

HTTP Server & REST API Cheatsheet

Lessons: [20 — HTTP Server Fundamentals](#) · [21 — Building a JSON REST API](#)

Examples: —

Covers: `net/http` server types, 1.22+ routing, request/response API, middleware, graceful shutdown

Legend: `[*]` = real Go API that the lessons have not covered yet

THE CORE TYPES

<code>http.Handler</code>	interface { <code>ServeHTTP(w ResponseWriter, r *Request)</code> }
<code>http.HandlerFunc(f)</code>	adapter: makes a func into a Handler
<code>http.ResponseWriter</code>	interface: <code>Header()</code> , <code>Write([]byte)</code> , <code>WriteHeader(int)</code>
<code>*http.Request</code>	everything about the incoming request
<code>http.ServeMux</code>	the router; it is itself a Handler
<code>http.Server</code>	the configured server (use this, not <code>ListenAndServe</code>)
(a middleware is just <code>func(http.Handler) http.Handler</code>)	

ROUTING (Go 1.22+)

<code>mux := http.NewServeMux()</code>	a fresh router
<code>mux.HandleFunc("GET /tasks", listTasks)</code>	METHOD + path in the pattern
<code>mux.HandleFunc("POST /tasks", createTask)</code>	
<code>mux.HandleFunc("GET /tasks/{id}", getTask)</code>	a wildcard segment
<code>mux.HandleFunc("GET /files/{path...}", ...)</code>	trailing wildcard: matches the rest
<code>mux.HandleFunc("/", notFound)</code>	the catch-all
<code>mux.Handle("/x/", http.StripPrefix("/x/", h))</code>	<code>[*]</code> mount a sub-handler
<code>r.PathValue("id")</code>	read a wildcard
<code>r.SetPathValue(k, v)</code>	<code>[*]</code> set one (useful in tests)
<code>"GET /tasks/{\$}"</code>	<code>[*]</code> <code>{\$}</code> anchors: match <code>/tasks/</code> exactly, not below it
(more specific patterns win; a conflict panics at registration, not at request time)	

READING THE REQUEST

<code>r.Method</code>	"GET", "POST", ...
<code>r.URL.Path</code>	the path, already cleaned
<code>r.URL.Query().Get("q")</code>	one query parameter ("" when absent)
<code>r.URL.Query()["tag"]</code>	<code>[*]</code> every value for a repeated parameter
<code>r.PathValue("id")</code>	a routing wildcard
<code>r.Header.Get("Authorization")</code>	one header, case-insensitive
<code>r.Header.Values("Accept")</code>	<code>[*]</code> all values
<code>r.Body</code>	an <code>io.ReadCloser</code> ; the server closes it for you
<code>r.Context()</code>	cancelled when the CLIENT disconnects
<code>r.RemoteAddr</code>	<code>[*]</code> host:port of the peer (the proxy, if there is one)
<code>r.Cookie("session")</code>	<code>[*]</code> → (<code>*http.Cookie</code> , error)
<code>r.FormValue("name")</code>	<code>[*]</code> parses the form body or query
<code>r.ParseMultipartForm(n) / r.FormFile("f")</code>	<code>[*]</code> file uploads
<code>r.Host / r.Proto / r.TLS</code>	<code>[*]</code> host header, version, TLS state

WRITING THE RESPONSE

```
w.Header().Set("Content-Type", "application/json")    headers BEFORE the body
w.WriteHeader(http.StatusCreated)                    status BEFORE the body, once
w.Write(b)                                           the body; implies 200 if you never set a status
json.NewEncoder(w).Encode(v)                       the JSON one-liner
http.Error(w, msg, code)                            plain-text error + status in one call
http.NotFound(w, r)                                 [*] 404 helper
http.Redirect(w, r, url, http.StatusSeeOther)       [*] 3xx
http.SetCookie(w, &http.Cookie{...})              [*] add a Set-Cookie header
w.(http.Flusher).Flush() [*] push what's buffered (SSE/streaming)
http.ServeFile(w, r, path)                          serve one file (path-traversal safe)
http.FileServer(http.Dir("public"))                 serve a directory
http.FileServerFS(fs) [*] serve an embed.FS
(after WriteHeader, changing headers does nothing – order is not optional)
```

STATUS CODES WORTH KNOWING

200 StatusOK	the default success
201 StatusCreated	a resource was created; set Location
202 StatusAccepted	queued, not done yet (background jobs)
204 StatusNoContent	success, no body (DELETE)
301 / 302 / 303 / 307	moved / found / see other / temporary
304 StatusNotModified	conditional GET hit the cache
400 StatusBadRequest	malformed syntax or invalid values
401 StatusUnauthorized	not authenticated (send WWW-Authenticate)
403 StatusForbidden	authenticated, not allowed
404 StatusNotFound	no such resource
405 StatusMethodNotAllowed	wrong verb; set Allow
409 StatusConflict	version conflict, duplicate key
415 StatusUnsupportedMediaType	wrong Content-Type
422 StatusUnprocessableEntity	syntactically fine, semantically wrong
429 StatusTooManyRequests	rate limited; set Retry-After
500 StatusInternalServerError	your bug – never leak the detail
502 / 503 / 504	bad gateway / unavailable / gateway timeout

THE SERVER

```
srv := &http.Server{
    Addr:           ":8080",
    Handler:        mux,
    ReadTimeout:    5 * time.Second,      whole request, including body
    ReadHeaderTimeout: 2 * time.Second,  [*] headers only – the Slowloris defense
    WriteTimeout:   10 * time.Second,     start of read to end of write
    IdleTimeout:    60 * time.Second,     keep-alive idle
    MaxHeaderBytes: 1 << 20,             [*] header size cap
}
srv.ListenAndServe()           blocks; returns http.ErrServerClosed on Shutdown
srv.ListenAndServeTLS(cert, key) [*] HTTPS
srv.Serve(listener)            [*] bring your own net.Listener
srv.Shutdown(ctx)              stop accepting, drain in-flight, then return
srv.Close()                    [*] hard stop, drops connections
http.ListenAndServe(":8080", mux) the toy form: no timeouts, no shutdown
(a server with no timeouts is a production incident waiting to happen)
```

GRACEFUL SHUTDOWN (the whole shape)

```
ctx, stop := signal.NotifyContext(context.Background(), os.Interrupt, syscall.SIGTERM)
defer stop()
go func() { if err := srv.ListenAndServe(); err != http.ErrServerClosed { log... } }()
<-ctx.Done()                                a signal arrived
shutCtx, cancel := context.WithTimeout(context.Background(), 10*time.Second)
defer cancel()
srv.Shutdown(shutCtx)                        drain, or give up after 10s
(then close the DB, flush logs – in that order, after Shutdown returns)
```

MIDDLEWARE

```
func Logging(next http.Handler) http.Handler {
    return http.HandlerFunc(func(w http.ResponseWriter, r *http.Request) {
        start := time.Now()
        next.ServeHTTP(w, r)
        slog.Info("request", "method", r.Method, "path", r.URL.Path,
            "dur", time.Since(start))
    })
}
handler = Logging(Recover(Auth(mux)))        outermost runs FIRST
Recover                                     defer recover() -> 500, keep the server alive
RequestID                                  generate an id, put it in ctx and the response header
Auth                                       validate, put the identity in ctx, or 401
CORS                                       set the headers, answer OPTIONS preflight
http.TimeoutHandler(h, d, msg)           a per-request deadline with a 503
(capture the status code by wrapping ResponseWriter in your own struct)
```

REQUEST-SCOPED CONTEXT

```
ctx := r.Context()                         cancelled when the client goes away
r = r.WithContext(ctx)                     middleware attaches values this way
type ctxKey struct{}                       an unexported key type – never a bare string
context.WithValue(ctx, userKey{}, u)       store
u, ok := ctx.Value(userKey{}).(*User)     retrieve
ctx, cancel := context.WithTimeout(r.Context(), 3*time.Second)  per-call budget
(only request-scoped data belongs in ctx – never config or dependencies)
```

JSON REQUEST/RESPONSE HYGIENE

```
r.Body = http.MaxBytesReader(w, r.Body, 1<<20)    cap the body FIRST
dec := json.NewDecoder(r.Body)
dec.DisallowUnknownFields()  reject typos and overposting
if err := dec.Decode(&in); err != nil { 400 }
in.Validate()                  your own method, or a validator library
w.Header().Set("Content-Type", "application/json")
json.NewEncoder(w).Encode(out)
never decode straight into a domain model    use a request DTO
never return the domain model                use a response DTO – it's your API contract
(one error shape for the whole API; see the API design sheet for problem+json)
```

THE CLIENT SIDE

```
http.Get(url) / http.Post(...)    convenience: uses DefaultClient, NO timeout
client := &http.Client{Timeout: 10 * time.Second}    always set one
req, _ := http.NewRequestWithContext(ctx, "GET", url, body)    [*] the right way
resp, err := client.Do(req)
defer resp.Body.Close()    ALWAYS, even when you ignore the body
io.Copy(io.Discard, resp.Body)    [*] drain it so the connection is reused
resp.StatusCode / resp.Header / resp.Body
&http.Transport{MaxIdleConnsPerHost: 100}    [*] the connection pool
(reuse ONE client; a new client per request leaks connections)
```

TRAPS & MEMORIZE

http.ListenAndServe	no timeouts – fine for a demo, never for production
WriteHeader after Write	ignored, with a "superfluous" log line
setting headers after Write	silently does nothing
forgetting resp.Body.Close()	leaks a connection per call
r.Context() for background work	cancelled the moment the response is sent
storing config in ctx	ctx is for request-scoped data only
string keys in ctx	collide across packages; use an unexported type
one global mux with no timeouts	a slow client can hold a goroutine forever
returning the domain struct	every field change becomes an API change
ignoring the Decode error	you get a zero-valued struct and no clue why
404 from the "/" catch-all	it matches everything, including real typos
handler panics	without Recover, one panic kills the process